

Temporal Reasoning under Masked Dates: Evaluating Local LLMs on Ordered Events and Era Estimation

IDS 570 - Text as Data

1 Introduction

Temporal language appears everywhere: in news articles, historical writing, fiction, and workplace documents. However, many NLP systems still treat documents mostly as static collections of words, even though many real tasks depend on understanding when events happened and how they relate to each other. Tasks like summarization, question answering, timeline construction, and misinformation analysis often require knowing what happened before what, and roughly when a narrative is taking place.

Large language models often seem fluent when discussing history, dates, and timelines. However, it is not always clear whether they are actually reasoning from the text or relying on memorized patterns and well-known timelines from training data. This project tries to separate temporal inference from simple memorization by removing explicit temporal expressions before giving the text to a model.

The main question is practical: when a model is given prose with dates and time expressions removed, can it still do three things? First, can it identify the important events and place them in the correct chronological order? Second, can it estimate the general era of the passage? Third, when a gold standard year range is available, can it give a reasonable calendar range? To test this, I built a reproducible evaluation pipeline using two difficulty tiers, local model inference, and simple string-based metrics that work well for a smaller student-scale project.

2 Corpus Selection and Suitability for NLP

2.1 Two tiers, two purposes

The evaluation uses two tiers. The easy tier is built from annotated corpora with established gold temporal relations. The hard tier is built from synthetic passages where the timeline is controlled and not present in any pretraining set. Together they let the same pipeline measure both how well a model uses real-world knowledge and how well it reasons from text alone.

Easy tier: TimeBank and MATRES. TimeML provides a representation for events, temporal expressions, and temporal relations in machine-readable XML. TimeBank applies that scheme to news text, and the loader extracts the document text and uses TLINK relations between event instances to derive a gold event order.

MATRES focuses on main-axis temporal relations between event mentions [3]. Instead of trying to label every possible event pair, it focuses on relations that are more central to the main timeline of a document. Pairwise labels such as **before**, **after**, and **equal** are merged into a graph, and a topological sort produces a linearized order when the graph is acyclic. Pairing the MATRES TSV with TempEval-3 source files [8] yields complete document examples that include both the surface text and a gold temporal ordering.

These two corpora form the easy tier because they consist of real news passages whose underlying events are often well represented in LLM pretraining. Even after masking, a model can sometimes recover the correct order from world knowledge. This makes the tier a useful upper bound: if a model cannot succeed here, it is unlikely to succeed on the harder tier.

Hard tier: synthetic passages. The hard tier uses a generator that produces short passages where the true timeline is stored separately from the surface text [cf. reading-style temporal benchmarks such as 4]. Names, places, and event content are constructed so that a pretrained model is unlikely to have seen the specific timeline before. This stresses whether the model is using genuine discourse reasoning or matching recognizable lexical patterns.

2.2 Appropriateness for NLP analysis

All sources are converted into a shared JSON format. Each item contains raw text, a gold chronological list of events, an optional gold year range, and metadata such as tier, corpus, and id. This common format matters because the same masking, prompting, batching, and scoring code runs across both tiers.

The documents are English prose at paragraph length, usually around 80 to 250 words. This length fits comfortably into local LLM context windows and is short enough for manual error analysis.

The task itself is structured prediction. The model returns a JSON object containing ordered events, an approximate era, and an estimated year range. Structured output is also a common way modern LLMs are used in applications, especially for tool use and evaluation. This setup is realistic while still being simple enough to grade automatically.

3 Historical and Contextual Background

3.1 From linguistic time to annotated corpora

Linguists have long separated ideas such as speech time, event time, and reference time when studying tense and aspect. Computational work in the early 2000s tried to turn these ideas into practical annotation schemes for English newswire text. TimeML was one important result of this effort. It provided tags for events, temporal expressions, and temporal signals [6].

TimeBank showed that human annotators could create reasonably stable event graphs based on text. Later shared tasks such as TempEval-3 standardized evaluation for temporal expression grounding, event detection, and temporal relation classification [8]. These efforts helped support later datasets such as MATRES, which focuses on temporal relations that are especially important for downstream tasks [3].

3.2 From relation classification to narrative understanding

Earlier temporal systems often treated the task as pairwise relation classification. Given two marked events, the model predicted whether one happened before, after, or at the same time as the other. Neural models improved performance, but many systems still did not fully use document-level context.

Reading-comprehension datasets later asked models to answer temporal questions in context, requiring integration of information across sentences [4]. This project is closer to open generation: the model proposes its own event descriptions and orders them. That is harder than choosing among candidate answers, but closer to how someone would actually use an LLM to draft a timeline from a document.

3.3 Masking as a stress test

Many public benchmarks include explicit calendar years or temporal clues in the input. If a model can see the year or date, it may not need much reasoning. To reduce this shortcut, the pipeline masks explicit dates and many relative time expressions before giving the passage to the model.

The default masking removes things like digit years, month-day patterns, decade phrases, and BC/AD markers. More aggressive masking can remove phrases like “last Tuesday,” but this can sometimes remove useful discourse anchors. For this reason, the report treats standard masking as the default setting.

4 Literature Review

Work on temporal reasoning in NLP has moved through three rough phases: structured annotation of news text, supervised relation classification, and prompt-based evaluation of large language models. Each phase shaped the design choices in this project.

Structured annotation. TimeML established the idea that temporal meaning should be grounded in the text through spans and document-level relations rather than described in free prose [5, 6]. The `EVENT`, `TIMEX3`, and `TLINK` tags gave annotators a vocabulary that was specific enough to support automatic systems. TimeBank applied this scheme to a corpus of news articles and showed that trained annotators could reach reasonable agreement on event identification and ordering. TempEval-3 then turned this annotation work into a shared task with held-out evaluation data, which made cross-system comparison meaningful for the first time [8]. The easy tier in this project inherits its gold ordering directly from these annotation traditions.

Refining what to annotate. A practical problem with TimeML-style annotation is that not every event pair in a document is equally informative. Many `TLINK` edges are redundant, and some are difficult to label consistently. MATRES responded to this by restricting attention to main-axis relations between events that are closer to the spine of the narrative [3]. The benefit is twofold: agreement is higher, and the resulting partial order tends to match what a human reader would actually reconstruct from the passage. This is why the MATRES loader in this project uses topological sort to derive an event sequence rather than scoring pairwise predictions directly.

Temporal commonsense work in the same period showed that even basic reasoning about typical event durations and frequencies remained difficult for strong neural systems [9], suggesting that surface-level cues alone were not enough.

As contextual encoders became standard, researchers began testing temporal understanding through more open-ended formats. TORQUE asked annotators to write questions whose answers required temporal reasoning over a passage, which forced models to combine event extraction with cross-sentence inference [4]. TimeQA pushed in a similar direction by asking time-sensitive questions where the correct answer changes depending on the period in question [1]. These benchmarks marked a shift from labeling annotated spans toward generating answers, which is closer to how downstream applications use temporal information.

With the arrival of large pretrained models, attention turned to whether these systems carry temporal knowledge in their parameters and whether they reason with it reliably. Time-aware language models showed measurable benefits from conditioning on the timestamp of pretraining text, suggesting that temporal context is a learnable signal rather than a fixed property [2]. Targeted probes have revealed both strengths and clear blind spots. Models often recall famous date-event associations but fail on lower-frequency facts and on questions that require composing two temporal facts together [7].

This project uses gold temporal orderings from the structured-annotation tradition (TimeBank and MATRES) and combines them with synthetic items in the spirit of recent LLM probing work, where controlling for memorization is essential. The masking step is a deliberate choice to push the evaluation away from surface lookup and toward discourse reasoning. The metrics are intentionally simple, with string-based scoring rather than full span alignment, because the goal is a transparent and reproducible course-scale benchmark rather than a leaderboard system.

5 Methods

5.1 Masking

The masking function applies a sequence of regular-expression substitutions. It removes numeric dates, month-day expressions, bare years, decade phrases, and BC/AD markers. An optional aggressive mode also masks relative weekday phrases. After masking, the resulting text is inserted into a fixed instruction template.

5.2 Prompting and decoding

The system prompt tells the model to return JSON with three fields: `ordered_events`, `approximate_era`, and `estimated_year_range`. The user prompt contains only the masked document. Models are accessed through a small client layer that supports Ollama and other backends configured through environment variables.

5.3 Metrics

The project compares the gold chronological event list to the model’s predicted event list. Let $g_1, \dots, g_k$ be the gold ordered event strings and $p_1, \dots, p_m$ be the model predictions.

Pairwise order accuracy. Each gold event is greedily matched to a predicted event using normalized sequence similarity. The default threshold is 0.45. For every pair of gold events that can be matched to predicted events, the metric checks whether the model placed them in the correct order. The final score is the fraction of matched pairs with correct ordering.

Mean best-match similarity. Each gold event is compared to all predicted event lines. The best fuzzy similarity score is recorded for each gold event, and these scores are averaged. This measures whether the model described the correct events, even if the order is wrong.

Year range overlap. If the gold answer includes a year range and the model predicts two integers, the pipeline computes a Jaccard-like overlap between the two inclusive ranges.

6 Experiments

6.1 Data and models

Easy-tier items come from TimeBank-style XML files and from MATRES rows paired with TempEval-3 source text. Hard-tier items come from the synthetic generator. Representative runs use one or more local Ollama-hosted models, such as `llama3.2`, `mistral`, or `qwen2.5`.

6.2 Procedure

For each model-document pair, the evaluator loads the raw text, masks temporal expressions, sends the prompt to the model backend, parses the JSON response, and computes metrics against the gold fields when available.

Some items may be skipped. For example, a MATRES row may be skipped if the matching TempEval text file is missing, and a TimeBank document may be skipped if its TLINK graph cannot be linearized. These cases are recorded with a reason and excluded from the averages. A tier filter allows separate evaluation on the easy or hard subset.

6.3 Reproducibility

Most randomness comes from model decoding. For local Ollama models, fixing temperature and seed settings can improve repeatability when the backend supports it. The plotting scripts use the saved evaluation JSON, so figures can be regenerated without re-running the models.

6.4 Figures from evaluation JSON

The evaluation JSON can be used to create several plots. One plot shows mean pairwise order accuracy by difficulty tier and model. Another shows the same information as a heatmap. A third plot compares per-document ordering accuracy to mean fuzzy match similarity. This helps identify cases where the model described the right events but put them in the wrong order.

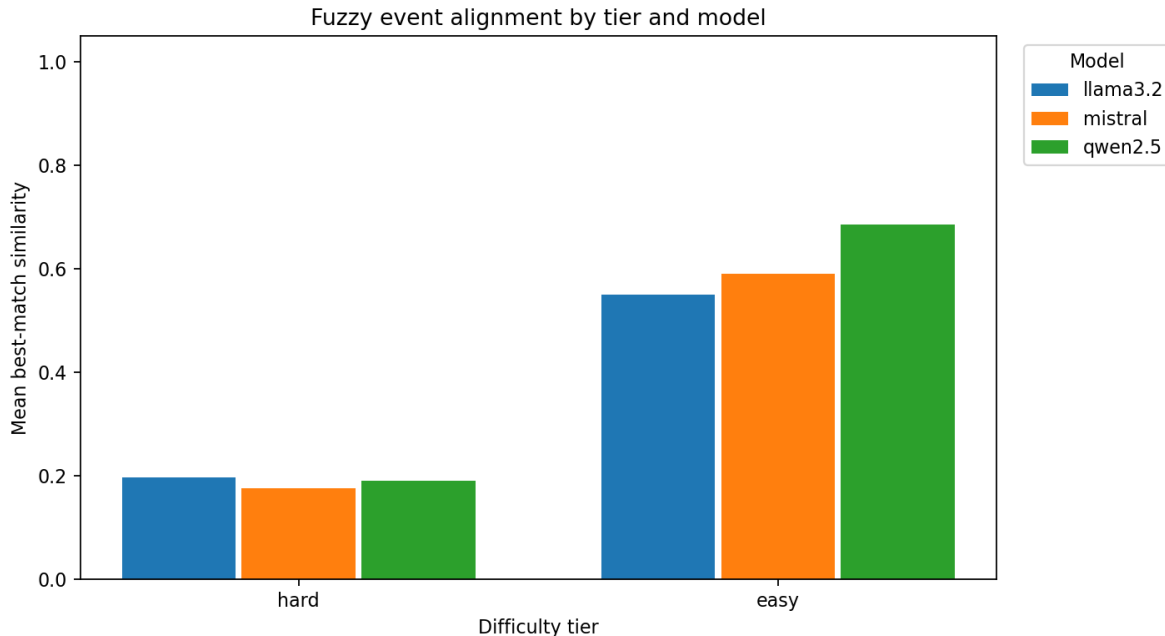


Figure 1: Mean match similarity.

7 Results and Discussion

On easy-tier items, models often perform well, especially when the events are widely represented in pretraining data. For example, a TimeBank passage about a high-profile political or business event can still be recognizable even after explicit years are removed. In these cases, the model may preserve the correct order because it already knows the broader sequence from training. MATRES items behave similarly: when the underlying news story is familiar, world knowledge can compensate for the masked dates.

The hard tier separates memorization from discourse reasoning more clearly. Synthetic settings remove the value of world knowledge because the names, places, and events are constructed for the evaluation. Performance variance across models grows here, and year-range overlap is often low or missing entirely when the model declines to commit to a range.

8 Limitations and Ethics

The synthetic generator introduces a possible train-test overlap issue if the same model family is used for both data generation and evaluation.

The masking method is based on regular expressions, so it is not perfect. It may miss unusual temporal expressions or remove too much information, making the passage harder to read than intended. Era labels and year ranges predicted by LLMs should also not be treated as authoritative historical claims. In this project, they measure agreement with a course gold standard, not historical truth.

8.1 Threats to validity

Construct validity. TimeML-derived orders depend on annotation guidelines and graph connectivity. MATRES linearizations can be somewhat arbitrary when the partial order is incomplete. Synthetic gold orders reflect the generator’s intent rather than independent annotation. Since model-predicted events are free-form strings, fuzzy matching is only an approximation of semantic correctness.

Internal validity. Models can differ in tokenizer behavior, chat templates, and context formatting, especially when accessed through Ollama. If different backends wrap prompts differently, small formatting differences may affect JSON validity and output quality.

External validity. The items are short English paragraphs and are not representative of entire news archives, multilingual data, or low-resource domains. The results should therefore be read as a case study in pipeline design rather than as a final leaderboard-style comparison.

9 Conclusion

This project implements a two-tier evaluation of temporal reasoning under masked dates. The easy tier draws on TimeBank and MATRES to anchor the benchmark in real annotated text, and the hard tier uses synthetic passages to control for memorization. The pipeline connects older TimeML-style annotation ideas with modern JSON-based LLM outputs, uses simple and interpretable metrics, and supports reproducible plots from saved model outputs.

Future work could improve the project by adding span-based scoring against TimeML gold annotations, expanding the synthetic generator to cover more narrative structures, and comparing open-weight models with commercial APIs under the same prompts.

The script reads the saved evaluation output and writes figures to the report figures folder. These generated images can then be copied or committed alongside the report before uploading to Overleaf. If the filenames differ, update the `\includegraphics` paths accordingly.

References

- [1] Wenhua Chen, Xinyi Wang, and William Yang Wang. A dataset for answering time-sensitive questions. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. doi: 10.48550/arXiv.2108.06314. URL <https://arxiv.org/abs/2108.06314>. arXiv:2108.06314.
- [2] Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics*, 10:257–273, 2022. doi: 10.1162/tacl_a.00459. URL <https://aclanthology.org/2022.tacl-1.15/>.
- [3] Qiang Ning, Zhili Feng, Subhro Roy, and Dan Roth. A structured learning approach to temporal relation extraction. In *Proceedings of the 56th Annual Meeting of the Association for*

- Computational Linguistics (Volume 1: Long Papers)*, pages 1287–1298, Melbourne, Australia, 2018. Association for Computational Linguistics.
- [4] Qiang Ning, Hao Wu, Dan Roth, Ziqi Feng, Vedant Narayan, and Apurva Joshi. TORQUE: A reading comprehension dataset for temporal reasoning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1278–1288, Online, 2020. Association for Computational Linguistics.
 - [5] James Pustejovsky, José Castaño, Robert Ingria, Roser Saurí, Rob Gaizauskas, Andrea Setzer, and Graham Katz. TimeML: A specification language for temporal and event expressions. 2003.
 - [6] James Pustejovsky, Kiyong Lee, Harry Bunt, and Laurent Romary. ISO-TimeML: An international standard for semantic annotation. In Nicoletta Calzolari, Khalid Choukri, Bente Maegaard, Joseph Mariani, Jan Odijk, Stelios Piperidis, Mike Rosner, and Daniel Tapias, editors, *Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC’10)*, Valletta, Malta, May 2010. European Language Resources Association (ELRA). URL <https://aclanthology.org/L10-1027/>.
 - [7] Qingyu Tan, Hwee Tou Ng, and Lidong Bing. Towards benchmarking and improving the temporal reasoning capability of large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14820–14835, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.828. URL <https://aclanthology.org/2023.acl-long.828/>.
 - [8] Naushad UzZaman, Hector Llorens, James Derczynski, James Allen, Marc Verhagen, and James Pustejovsky. SemEval-2013 task 1: TempEval-3: Evaluating time expressions, events, and temporal relations. In *Proceedings of the Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, Atlanta, Georgia, USA, 2013. Association for Computational Linguistics.
 - [9] Ben Zhou, Daniel Khashabi, Qiang Ning, and Dan Roth. “going on a vacation” takes longer than “going for a walk”: A study of temporal commonsense understanding. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3363–3369, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1332. URL <https://aclanthology.org/D19-1332/>.